

flocks-egress 内部说明（给我们自己看，不进交付包）

客户看的是 README.md（安装 / 使用 / 排障）。这里放机制、边界和验证记录，改代码前先读这个。

目录

- 1. 机制
- 2. DNS 两种模式
- 3. 容器流量
- 4. 引擎不在时
- 5. 边界
- 6. 已验证的范围
- 7. 仓库里的其他东西

1. 机制

setup-a.py（Python 3.9 标准库）装两个 systemd 服务：flocks-egress-rules（nft 规则，DefaultDependencies=no，开机在 network-pre.target 之前就位）和 flocks-egress（引擎 mihomo，用户 flocks-egress，只保留 CAP_NET_ADMIN，PartOf 规则服务）。两张 nft 表：常驻的 inet flocks_egress 和随引擎进程启停的 inet flocks_egress_engine（引擎 ExecStartPre 加载、ExecStopPost 删除）。

分流按目标 IP：direct4 / direct6 集合 = RFC1918、回环、链路本地、100.64/10、组播、DIRECT_CIDRS、代理 B 的地址（域名则装时解析）、内网 DNS 地址、本机自己的地址。命中就 return；其余公网 TCP 在 output nat 链 redirect to :7892，公网 UDP、公网 IPv6 TCP 在 output filter 链 reject。范围规则：SCOPE=host 是 meta skuid <引擎 uid> return（放过引擎自己，防回环），SCOPE=user 是 meta skuid != <flocks uid> return。

机器 A 的任意进程（flocks、dnf、curl...）



引擎配置用 mihomo 的 `listeners:: redir-in` (type redir, 端口 7892, 开着容器接管时监听 0.0.0.0, 否则 127.0.0.1) 和 `mixed-in` (type mixed, 7890, 永远 127.0.0.1, 没有认证, 所以不能开到外面)。sniffer 开着 (HTTP 80 / 8080-8880, TLS 443 / 465 / 993 / 8443), 纯 IP 目标也能以域名形式交给 B。嗅探端口只能加「客户端先说话」的: 25 / 587 / 143 / 110 这类服务端先发 banner 的不能加, sniffer 读 1 秒读不到数据会把连接关掉。SSH 这类没有 SNI 的协议在 redir-host 下只能 `CONNECT ip:22`。凡是 TCP 都会被转给 B, 能不能通看 B 放行哪些端口 (squid 默认只有 443)。不用 `external-controller`。

安装流程: 预检 (root、systemd、nft) → 解析参数 (命令行 (`--proxy` 或位置参数, 地址可以只写 主机:端口, `parse_proxy` 补 `http://`) > `--config` > `/etc/flocks-egress/egress.conf` > 同目录 `egress.conf` > 机器已有的代理设置: 环境变量 → `/etc/environment` → `/etc/profile.d/*.sh` → `dnf.conf` / `yum.conf` 的 `proxy=` → `/etc/wgetrc`, `detect_system_proxy()`, 取到的值解析不了就报一句来源然后忽略 > stdin 是终端就问三句 (地址 / 账号 / 密码, `ask_proxy_interactively()`; 答错重问最多三次, Ctrl-D 当退出; 不是终端直接报错, 脚本和管道里不会卡住) → 探测 DNS 模式 → **子进程真的经代理 CONNECT 一次** (`PROXY_PROBE_SRC`, `http` / `https` / `socks5` 三种协议、407 / 403 / 证书 / 连不上分别报错) → 建用户 → 渲染 `config.yaml` / `rules.nft` / `engine.nft` 并各自自检 (`mihomo -t`、`nft -c`) → 落盘 (`egress.conf` 0600、`settings.json` 不含密码、`config.yaml` 0640 root:flocks-egress) → 单元 + `nftables.service.d/flocks-egress.conf` drop-in → 启动 → `check`。全程幂等。探测子进程以引擎用户跑 (规则里放过的 uid), 透明代理运行中再探测不会被自己接管; 代码通过 `python3 -c` 传, 密码走 stdin。

`nftables.service` 的 `start` / `reload` / `restart` 会 `flush ruleset`: drop-in 在它之后 `systemctl --no-block try-reload-or-restart flocks-egress-rules`, 规则单元的 `ExecReload` 是 `flocks-egress _rules-reload` (重载常驻表, 引擎在跑就连引擎表一起补)。`systemctl stop nftables` 到再 `start` 之间规则是空的。

2. DNS 两种模式

`DNS_MODE=auto` 安装时直接问内网 DNS (`DNS_PROBE_SRC`, 不走系统解析器) 能否解析一个公网域名:

- 能 → `redir-host`: 完全不碰 DNS, 程序拿真实 IP, 纯按 IP 分流, 引擎靠 SNI / Host 嗅探拿域名。不需要域名名单。允许本机 DNS 转发器。
- 不能 → `fake-ip`: 引擎表里的 `dns_nat` (`output nat`, `dstnat - 10`, 排在分流链前) 把本机 53 转给引擎 1053, 公网域名返回 198.18.x.x 占位、连接进引擎后按域名交给 B 解析; 内网域名靠 `resolv.conf` 的 `search` 域 / `DIRECT_DOMAINS` 识别返回真实 IP。
`SCOPE=host` 时 `DNS_SERVERS` 不能是本机地址 (转发器上游那一跳会被接管形成回环, 代码

拒绝)。极偶发 DNS 绕过：引擎用户 30 秒内用同一源端口向同一 DNS 查过时，别的进程那一次会沿用旧连接跟踪拿到真实答案，无害（TCP 仍被接管），check 因此连查 3 次。

3. 容器流量

Docker / rootful podman 的 bridge 网络有自己的网络命名空间，包从网桥转发出来、不经 output 链。CONTAINERS=auto（host 范围开、user 范围关——转发包没有 uid 可认）时多三条链：egress_input（filter input, priority filter - 1: iifname lo return、meta iifkind "bridge" return，其余接口到 7892 / 1053 的 drop——因为 redir 口和 fake-ip 的 DNS 口这时监听 0.0.0.0）、egress_prerouting（nat prerouting: meta iifkind != "bridge" return、fib daddr type local return、direct 集合 return、redir-host 下发往公网 DNS 的 53 dn timer ip to <内网 DNS>、公网 TCP redirect）、egress_forward（forward filter: 公网 UDP / IPv6 TCP reject）；fake-ip 时引擎表再加 dns_prerouting。

上联网桥：detect_uplink_bridges() 把「默认路由所在的网桥」和「成员口不是 veth 的网桥」（物理网卡、bond、vlan、虚机 tap）识别为上联口，四条容器链渲染 iifname { ... } return 排除；check 每次重新探测，和 settings.json 不一致就 WARN。redir 口监听 0.0.0.0 的可滥用性：目标来自 SO_ORIGINAL_DST，没被 nft 转过的连接拿到的是自己的地址，客户端指定不了目标；7890 只在回环。

不覆盖：macvlan / ipvlan、没有网桥的 CNI（Calico 类）、OVS 网桥；rootless podman / docker 的流量出自宿主用户进程，按 uid 走 output 链，本来就在覆盖内。flocks 的 Docker 沙箱默认 network: none。Docker 装在有 firewalld 的机器上会把 docker0 放进它的 docker zone，容器到本机 7892 是通的；别的容器运行时要自己确认。fake-ip 下容器自建的 DNS 服务会被绕过；redir-host 下只改写发往公网 DNS 的查询。

4. 引擎不在时

ENGINE_DOWN（默认 direct，用户 2026-09-22 定的）：

- direct：分流链、集合、容器 prerouting / forward 都在引擎表里，引擎一停整张表卸掉，公网 TCP、UDP 都直连兜底，引擎回来又接管。常驻表只剩 egress_input 和（fake-ip 时）egress_fakeip_guard / egress_fakeip_forward——程序手里已经解析到的 198.18.x.x 占位地址在引擎停掉后没处可去，这两条让它立刻 RST 而不是挂到超时；引擎在跑时这种包早在 nat 里改成了 127.0.0.1，碰不到这条。
- reject：分流链在常驻表，引擎不在时公网 TCP 转到没人听的 7892 → RST，立刻失败、不漏成直连（fail-closed）；容器同样。
- direct + fake-ip 还有一半兜不住：引擎重启的那 2 秒里 DNS 走真实的内网 DNS，而这种环境的内网 DNS 对公网域名答 NXDOMAIN，带负缓存的解析器（systemd-resolved、nscd、

Java 默认 10 秒) 在引擎回来后还会按缓存继续报「解析失败」直到过期。这是 direct 的固有代价, `egress_fakeip_guard` 只管「已经拿到 198.18.x.x」那一半。

两种模式下的前提都是常驻表加载成功; `nft -f` 失败则引擎 (Requires=) 也起不来, `check` 两服务都 FAIL。UDP 为什么只有拒绝 / 直连: `nft redirect` 查不回 UDP 报文的原目标 (TCP 靠连接跟踪能查), 透明接管 UDP 得换 TPROXY + 策略路由 (`fwmark + ip route local ... table 100`); HTTP 代理带不了 UDP; `flocks` 用不到公网 UDP。要做的话只在 SOCKS5 代理下有意义。

5. 边界

- `check` 的「公网走代理」在 `ENGINE_LOG_LEVEL=info/debug` 下以引擎日志行为证据 (cursor 增量读 journal, 不整段拉), warning 以上改凭 `nft redirect` 计数 (只做正向证据, 别的进程也会推高它), 「例外用户不被接管」降为 WARN。
- `ENGINE_DOWN=direct` 且引擎不在时 `check` 把「nft 分流规则」「公网走代理」报 WARN (兜底状态), 「内网直连」只按 http 判。
- 扫描类工具对公网的结果不可信 (连接被接管交给 B)。
- 早期 2.0.0 构建的引擎表叫 `flocks_egress_dns` (文件 `dns.nft`), `drop_legacy_table()` 在 `install / reload / rollback` 里顺手删。
- `meta iifkind` 需要内核 5.1+ / `nftables` 0.9.1+, `inet` 表里的 `nat` 需要内核 5.2+; RHEL 9 的 5.14 按版本推断满足, e2e 跑在宿主 6.8 内核上, 没在 5.14 上实跑。

6. 已验证的范围

`tests/e2e-centos9.sh` 在 Docker 里搭四台机器: 跑 systemd 的 CentOS Stream 9 「机器 A」和「机器 B」、内网 web、内网 DNS (`dnsmasq`)。B 上不装本包任何东西, 扮演客户现成的代理: `squid` (3128 不认证、3129 basic 认证, `SSL_ports` 放开 80/443)、`socat` 用自签证书给 `squid` 套一层 TLS 当 https 代理、`tests/fixtures/socks5-proxy-on-b.sh` 用包里的 `mihomo` 起一个带认证的 HTTP+SOCKS5 (3130)。容器流量用网桥 + 网络命名空间 + `veth` + `MASQUERADE` 模拟 Docker bridge; 上联网桥用「把 `eth0` 挂进网桥、IP 和默认路由搬上去」模拟。

覆盖: 参数校验与 `dry-run`、代理探测六种失败、`auto` 两种探测结果、一行命令安装、文件权限与引擎权限、`host` / `user` × `redir-host` / `fake-ip` 场景 (HTTPS 与 80 端口 HTTP 经 B 且在 B 日志确认、`aiohttp` / `httpx`、DNS 行为、例外用户、内网域名 / IP 直连、别的机器连 A 的监听端口、公网 UDP 拒绝、纯 IP 目标 SNI 嗅探)、严格 DNS 环境 (内网 DNS 解析不了公网)、容器 (HTTPS 经 B 且引擎日志源地址是容器 IP、内网直连、公网 UDP 拒绝、7890 只在回环而局域网连 7892 被丢、`fake-ip` 下容器 DNS 被接管、`user` 范围不接管)、上联网桥识别与排除、默认 `direct` 的兜底 (停引擎后本机与容器直连、UDP 不再被拒、内网照常、引擎回来与 `kill -9`

后重新经 B)、`reject` 的 fail-closed (含容器)、fake-ip + direct 下缓存占位地址快速拒绝、四种代理写法 (squid 认证、socks5、https 自签、https 导入 CA；账号错误的重装被挡且原配置不动)、`ENGINE_LOG_LEVEL=warning` 的验收判据、`nftables.service start / reload / restart` 后自愈、容器重启后两服务自启且规则先于 `network-pre.target`、`firewalld` 共存、`systemd` 单元校验、rollback、残留拒绝、重装。

交付目标是 x86_64，验证以它为准；arm64 只是 Apple Silicon 开发机上容器跑得快 (10 分钟对 35 分钟)，用来快速迭代，包放 `dist/dev-arm64/`，不交付。改了 `setup-a.py` 之后的顺序：
`pytest` → 重打包 → **x86_64 e2e (QEMU 虚拟机，`DOCKER_CONTEXT=colima-x86`)** → 有空再跑 arm64。

环境	结果
x86_64 (交付目标)：CentOS Stream 9 容器 (systemd 作 PID 1)，跑在 QEMU 全系统模拟的 Linux 6.8 虚拟机里，用交付的 x86_64 包 + .run 安装	160 / 160
arm64 (开发机)：CentOS Stream 9 容器，宿主内核 Linux 6.8，用 dev-arm64 包 + .run 安装	160 / 160
<code>tests/test_setup_a.py</code> ：参数文件 / 代理 URL / 网段 / 三份渲染 / 现成代理设置解析 / 终端提问 (开发机 pytest)	42 / 42

跑了什么、结果、交付包校验和记在 `tests/results/README.txt` (原始日志只留在跑测试的机器上，不进仓库)。没验证的：物理机 / 客户虚拟机上的 CentOS 9、真实 Docker、客户真实的代理产品、SELinux enforcing、上联网桥判据里「成员口不是 veth」那条分支 (容器里 eth0 是 veth，只走到了默认路由那条)。跑测试的两个坑：`colima start` 另一个 profile 会切走 docker context (一律显式 `DOCKER_CONTEXT=`)；e2e 跑着的时候不能改 `tests/e2e-centos9.sh` (bash 边读边执行)。x86 VM 上读 nft 表的断言偶发拿到空输出，对这类断言重试 3 次。

7. 仓库里的其他东西

方案定稿过程中试过的另两条路都已经删掉，不要再找：B 侧一键装代理的 `extras/relay/` (客户 B 已经有代理，B 上不装东西) 和只管 flocks 进程、靠写 `.env` + `--no-proxy` 名单分流的 `extras/lite-client/` (接不了 Node MCP、浏览器、邮件、SSH 这些不读环境变量的流量，被整机接管替代)。e2e 里 B 上的 SOCKS5 代理改由 `tests/fixtures/socks5-proxy-on-b.sh` 起 (40 行，只是夹具)。

- `tools/fetch-mihomo.sh` (固定 v1.19.31, sha256 校验)、`tools/make-bundle.sh` (只打 A 侧，含 README.pdf / THIRD_PARTY_NOTICES；默认出交付用的 x86_64 包到 `dist/`，`--arch arm64` 出开发测试用的到 `dist/dev-arm64/`；每个架构同时产出 `.tar.gz` 和自解压单文件 `.run` ——一段 shell 头 + `__ARCHIVE_BELOW__` + 同一个 tar.gz，

解压到 `/opt/.flocks-egress-run.XXXXXX` (不用 `/tmp`: 客户常把它挂成 `noexec`, 引擎自检跑不起来; `setup-a.py` 自己也会在 `mihomo -t` 碰到 `PermissionError` 时先把二进制拷到 `/opt/flocks-egress/bin/mihomo.tmp` 再自检)、按 `tar` 清单删掉; 第一个参数不是子命令 (没给、地址、`--proxy` 之类) 就一律当 `install`, `help / -h` 在解压前就打用法, 所以客户只要 `sudo bash xxx.run` 或 `sudo bash xxx.run 10.0.0.5:3128`)。测试里模拟人在终端答题用 `tests/pty_drive.py` (等提示出来再敲; `getpass` 会先清输入队列, 一次性灌进去的密码会丢)。